

README

Manuscript title: Estimating Pair-Specific Network Effects in Binary-Action Games: Two-Sided Markets via Restricted Boltzmann Machines

Authors: Tetsuya Hoshino and Romans Panch

Manuscript ID: MS-BST-2025-01899.R2

Corresponding author: Tetsuya Hoshino

Estimation: Google Colab + NVIDIA Tesla T4 GPU

A GPU is required for practical estimation.

Figure redraws: use saved results; no GPU required.

1 Overview

This package contains the code, simulated data, true parameters, and saved estimation histories for Section 5.3, Figures 3 and 4. The authors ran the experiments in Google Colab. **Estimation requires a GPU for practical execution.** Section 5 gives Colab instructions for redrawing the figures from saved results and rerunning estimation. Figures 1, 2, and 5 are conceptual illustrations supplied with the manuscript; the theoretical appendix has no computational results.

2 Data availability and provenance

All data used in the numerical experiments are included. The authors generated binary actions from an RBM using seed 42, standard-normal true parameters, and 500 Gibbs iterations. Figure 3 uses 100,000 observations with 11 agents on each side; Figure 4 uses 500,000 observations with 15 agents on each side. No external or restricted data are required.

The following JSON files contain the original observations and estimation histories. Paths are relative to `replication_package/`.

Observations and true parameters:

`data/figure3/synthetic_data_N11x11_D100000_B1000.json`

`data/figure4/synthetic_data_N15x15_D500000_B1000.json`

Figure 3 estimation histories: contrastive divergence (CD) and maximum likelihood (ML), respectively.

`results/figure3/training_data_N11x11_D100000_B1000_cd.json`

`results/figure3/training_data_N11x11_D100000_B1000_ml.json`

Figure 4 estimation history: CD.

`results/figure4/training_data_N15x15_D500000_B1000_T7200_cd.json`

3 Variable dictionaries

All datasets use UTF-8 JSON. Here, n_1 and n_2 are the numbers of agents on each side, and D is the number of observations. The rows of `data1` and `data2` correspond to the same observations.

Observation files

Field	Definition
<code>data_parameters.n1</code>	Agents on each side: 11 each (Figure 3) or 15 each (Figure 4).
<code>data_parameters.n2</code>	
<code>data_parameters.data_size</code>	Observations: 100,000 (Figure 3) or 500,000 (Figure 4).
<code>data_parameters.gibbs_step</code>	Gibbs iterations for data generation: 500.
<code>true_parameters.W</code>	True $n_1 \times n_2$ matrix of pair-specific interaction parameters.
<code>true_parameters.b1</code>	True bias vectors of lengths n_1 and n_2 .
<code>true_parameters.b2</code>	
<code>data1</code>	$D \times n_1$ and $D \times n_2$ matrices of binary actions (0 or 1).
<code>data2</code>	

Estimation history files

Field	Definition
<code>data_parameters.n1</code>	Market dimensions and observation count, as above.
<code>data_parameters.n2</code>	
<code>data_parameters.data_size</code>	
<code>data_parameters.batch_size</code>	Observations per training batch: 1,000.
<code>train_parameters.training_method</code>	<code>contrastive_divergence</code> or <code>maximum_likelihood</code> .
<code>train_parameters.time_limit</code>	Training limit: 7,200 seconds, checked after each epoch.
<code>train_parameters.cd_step</code>	Gibbs steps per CD update: 1; unused for ML.
<code>learning_rate</code>	SGD learning rate: 0.1.
<code>losses</code>	Initial record, followed by one record per completed epoch.
<code>losses[].epoch</code>	Completed epochs and elapsed training seconds; initially zero.
<code>losses[].time</code>	
<code>losses[].mse_W</code>	Mean squared error (MSE) relative to the corresponding true parameters.
<code>losses[].mse_b1</code>	
<code>losses[].mse_b2</code>	
<code>losses[].mse_Wb</code>	MSE over all parameter entries; the plotted outcome.
<code>losses[].avg_epoch_loss</code>	Average post-update total-parameter MSE across batches; initially the total MSE.
<code>losses[].learning_rate</code>	Optimizer learning rate; absent from the initial record.

Histories store MSE and timing summaries. The figures plot minutes against MSE on a log scale.

4 Computational requirements

Environment. Use Google Colab with an NVIDIA Tesla T4 GPU (16 GB VRAM), which is the environment used by the authors. At the paper’s sample sizes, CPU estimation is too slow for practical use. Redrawing figures from saved histories does not rerun estimation and requires no GPU.

Software. Python with PyTorch, NumPy, Matplotlib, and tqdm, supplied by Colab. The short Colab T4 check used Python 3.13.15, PyTorch 2.11.0+cu128, CUDA 12.8, and NumPy 2.1.3; Matplotlib and tqdm versions were not recorded. Software versions from the original paper runs were not recorded.

Runtime. Redrawing takes seconds. Each estimation method runs for about two hours: Figure 3 uses ML and CD, and Figure 4 uses CD only, totaling about six hours plus data generation. Input and result files occupy approximately 90 MB uncompressed.

Randomness. The code sets seed 42 for Python, NumPy, PyTorch, and CUDA. GPU speed and library versions can affect the number of epochs completed within the time limit and therefore the time-based curves. The saved histories contain the full runs used in the paper.

5 Code

`run_all.py` is the master script; it uses `code/original_model.py` for the model and estimation. The included `code/HoshinoPancs_Replication.ipynb` is the original notebook. Its title and documentation were updated on September 23, 2026; the code and saved outputs were unchanged. Use the master script for the steps below.

Prepare Google Colab

Open a new Colab notebook and select a T4 GPU under Runtime > Change runtime type. Run this cell and upload `MS-BST-2025-01899.R2_replication.zip`:

```
from google.colab import files
import zipfile

uploaded = files.upload()
archive = next(name for name in uploaded if name.endswith(".zip"))
with zipfile.ZipFile(archive) as package:
    package.extractall(".")
%cd replication_package
```

Redraw Figures 3 and 4 from saved results

After the setup above, run:

```
!python run_all.py
```

This reads the three saved histories and writes `Figure_3.pdf`, `Figure_4.pdf`, and PNG copies to `outputs/<run-id>/`, together with `run.log` and `environment.json`. Curves use the archived times and MSE values; fonts and layout may differ from the manuscript.

Generate data and rerun estimation (GPU required)

To rerun the experiments after the same setup, run:

```
!python run_all.py --mode estimate --figure all --device cuda \  
  --data-source generate --time-limit 7200 --seed 42
```

This generates observations, runs ML then CD for Figure 3 and CD for Figure 4, and saves data, histories, and plots under `outputs/<run-id>/figure3/` and `figure4/`. New output directories are created for each run.

Save outputs

Download the output folder before ending the Colab session:

```
import shutil  
shutil.make_archive("replication_outputs", "zip", "outputs")  
files.download("replication_outputs.zip")
```